

1 Approach Comparison - Art Haxholli

1.1 SMT2020 / HVLM Dataset

Terminology I renamed the dataset’s terms to match the Digital Reference: **Tool** to **Equipment**, **Step** to **Operation**, and **Area** to **Fab**. Each rename was recorded in an `rdfs:comment` so that the mapping back to the Excel columns remains available.

Fatos and Olti both kept the dataset’s original vocabulary: **Tool**, **Tool.Group**, **Process.Step**, and **Area**. DR alignment makes the model easier to merge with other DR-derived work, while dataset-native naming makes it easier to compare directly with the source spreadsheet.

Hierarchies Each approach handled the dataset structure differently.

Dataset feature	Art	Fatos	Olti
Batching / cascading tools	Subclasses of Equipment.Group	Boolean data properties and a Batch class	Data properties
Lot priority tiers	priority and isSuperHotLot data properties	Subclasses Hot.Lot , Regular.Lot , and Super.Hot.Lot	Data properties
Downtime events	PM.Event and Breakdown.Event as siblings	Both subclasses of a shared Downtime class	Sibling classes

I turned a tool flag into subclasses but left lot priority as data, while Fatos did the reverse. Fatos’s **Downtime** class provides a clean structure because the two event types, PM and Breakdown, share the same TTF/TTR structure. The common parent makes this explicit.

Location. I introduced a separate **Location** class taken from the Digital Reference definition. Fatos kept **Area** as a class and represented tool-group location using a data property. Olti dropped the location concept and gave **Transport** a range of **Tool.Group** directly.

Instances. Fatos populated the ontology from the Excel dataset using six Python scripts checked into the repository. Olti’s model and mine are schema-level. Fatos’s approach demonstrates that the schema works with the actual data. Since generation is scripted, it can also be rerun whenever the source data changes.

Documentation I annotated every class and property with the reasoning behind it, including DR classes that I considered and rejected. Olti provided a definition for all ten of his classes. Fatos used `dcterms:description` on his classes, with several descriptions explaining the manufacturing meaning rather than only the meaning of the corresponding spreadsheet column. This gives his documentation a more physical, manufacturing-oriented framing than my column-oriented notes.

1.2 Ferroelectric HfO₂ Memory

1.2.1 Fatos

Fatos modelled one specific device: a 1T1C FeRAM based on the Okuno capacitor-under-bitline lineage and fabricated on 130 nm CMOS.

His model includes concrete parameters such as: 64 kbit array, 8 nm HZO, 30–50 nm TiN electrodes, RTA at 500 °C for 30 s, endurance above 10¹¹ cycles.

The diagram is divided into architecture, process steps, materials, and equipment. It also includes a class-definition table and a numbered source list with explicit source flags.

1.2.2 Olti

Olti modelled the wider ferroelectric field. His diagram begins with ferroelectrics in general, including perovskites, multiferroics, and hydrogen-bond ferroelectrics, with examples such as barium titanate and Rochelle salt represented as individuals.

HfO₂ deposition is placed inside a generic CMOS-like process flow.

His model also contains a branch that neither Fatos nor I included: a **Quality_Check** class containing polarization, XRD, thickness, retention, endurance, and leakage tests. These are connected to the output through **validates** and **verifiedBy** relationships.

His presentation also connects the work to UPaaS and Europe’s memory-technology gap.

1.2.3 Art

I modelled the literature rather than one device or the entire ferroelectric field.

The model is based on ten sources read in full. Every class and property contains a source tag down to section level. Eight process flows are represented as ordered individuals, together with a register of twenty-one contested items.

The diagram contains the complete class hierarchy together with its object and data properties. Uncertainty identified in the specification is preserved rather than being removed during modelling.

1.2.4 Comparison

Three main observations can be made across the three approaches:

- All three independently divided the domain into device structure, process steps, materials, and equipment.
- All three handled sources cautiously, but at different granularities: Fatos flagged sources per citation, I tagged them per ontology entity, and Olti cited them per slide.
- Depth and breadth were handled differently. Fatos provides concrete process values but little generality, I provide generality across ten papers, and Olti provides the broadest framing but the least technical detail.

Instances. Both Fatos and I included instance-level data, but in different ways.

Fatos placed parameter values directly on the diagram for the specific device being modelled.

My model uses a separate worked example based on Flows A and B1. It shows which classes each process step instantiates, which classes remain deliberately empty because the source does not provide the required information, and where the source contains contradictions.

1.3 What I Would Take From Each Approach

1.3.1 From Fatos

Fatos’s Python-script approach shows that generating ontology instances directly from the source spreadsheet, with the scripts stored in the repository, turns the model from a proposal into something testable.

It also exposes structures that may not be obvious during manual modelling, such as the step-to-step chain across a complete manufacturing route.

My FeHfO₂ instance example makes a similar argument manually for two process flows, but programmatic generation is what allows the approach to scale.

1.3.2 From Olti

The quality-control branch is a genuine gap in my FeHfO₂ model.

I already have **Measurement_Method** and **Measurement** classes, but they exist beside the process flow. Olti instead models verification as an explicit stage that the output passes through, which provides a clearer representation of process validation.